

Model Validations

In **Django**, **Model Validation** is mainly divided into **4 types**:

These are **Django's built-in validators** that are applied directly to model fields.

```
from django.db import models
from django.core.validators import MinLengthValidator, MaxValueValidator

class Student(models.Model):
    name = models.CharField(max_length=50, validators=[MinLengthValidator(3)])
    age = models.IntegerField(validators=[MinValueValidator(18),
    MaxValueValidator(60)])

    def save(self, *args, **kwargs):
        self.full_clean() # validation trigger
        super().save(*args, **kwargs)
```

note max_length controls database storage, MaxLengthValidator controls validation logic.

Validator	Used For
MinLengthValidator	Minimum characters
MaxLengthValidator	Maximum characters
MinValueValidator	Minimum number
MaxValueValidator	Maximum number
EmailValidator	Email format
RegexValidator	Pattern matching

2 Custom Field-Level Validation

When you need **custom rules**, you can create your own validator function.

Example:

```
from django.core.exceptions import ValidationError

def validate_name(value):
    if not value.isalpha():
        raise ValidationError("Name should contain only alphabets")

class Student(models.Model):
```

```

name = models.CharField(
    max_length=50,
    validators=[validate_name]
)
def save(self, *args, **kwargs):
    self.full_clean()    # validation trigger
    super().save(*args, **kwargs)

```

→ This validation checks **only one field** at a time.

3 Model-Level Validation (`clean()` method)

Used when validation depends on **multiple fields together**.

Example:

```

from django.core.exceptions import ValidationError
from django.db import models

class Student(models.Model):
    age = models.IntegerField()
    roll = models.IntegerField()

    def clean(self):
        if self.age < 18 and self.roll > 100:
            raise ValidationError(
                "Students under 18 cannot have roll number greater than 100"
            )
    def save(self, *args, **kwargs):
        self.full_clean()    # validation trigger
        super().save(*args, **kwargs)

```

→ The `clean()` method validates the **entire model**.

4 Database-Level Validation (Constraints)

These validations are enforced at the **database level**, so they cannot be bypassed.

Example:

```

from django.db import models

class Student(models.Model):
    email = models.EmailField(unique=True)

```

OR

```
class Student(models.Model):  
    class Meta:  
        constraints =  
[models.CheckConstraint(check=models.Q(age__gte=18), name="age_greater_than_18")]
```

full_clean()

- └─ clean_fields() Field-level validation
- └─ clean() Model-level validation
- └─ validate_unique() validate_unique()

full_clean() call automatically

Place	Call automatically?
Django Admin	✓
ModelForm	✓
DRF Serializer	✓
.save()	✗
.objects.create()	✗

full_clean() is a built-in Django method that executes field, model, and uniqueness validations together.

"full_clean()" is a built-in Django method that performs all validations together — it validates individual fields, runs model-level validation, and checks uniqueness constraints.